

```

1 #=====
2 # MIT 6.046J 代码精读 • PS7 - 网络流应用
3 # 源文件: applications.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS7 applications built on the max-flow solver.
7
8 7-1 dynamic_maxflow_update : recompute max flow after changing one edge's
9     capacity, in  $O(k*(V+E))$  where  $k = |\text{old\_cap} - \text{new\_cap}|$ , by pushing/retracting
10    exactly k augmenting units instead of recomputing from scratch.
11 7-2 edge_disjoint_paths : k edge-disjoint  $s \rightarrow t$  paths (via a super-source).
12 7-3 food_truck_assignment : minimize vouchers = maximize assigned customers, a
13    bipartite b-matching solved by max flow.
14 """
15 from __future__ import annotations
16
17 from collections import deque
18 from typing import Dict, List, Optional, Tuple
19
20 from maxflow import Edge, MaxFlow
21
22
23 # -----
24 # 7-1: dynamic max-flow after a single edge capacity change
25 # -----
26 def _augment_one(mf: MaxFlow, s: int, t: int) -> int:
27     """Find one augmenting path (BFS) and push its bottleneck; return the amount."""
28     parent_edge: List[Optional[Tuple[int, int]]] = [None] * mf.n
29     parent_edge[s] = (-1, -1)
30     q = deque([s])
31     while q:
32         u = q.popleft()
33         if u == t:
34             break
35         for idx, e in enumerate(mf.graph[u]):
36             if e.residual() > 0 and parent_edge[e.to] is None:
37                 parent_edge[e.to] = (u, idx)
38                 q.append(e.to)
39     if parent_edge[t] is None:
40         return 0
41     # bottleneck
42     bottleneck = float("inf")
43     v = t
44     while v != s:
45         u, idx = parent_edge[v]
46         bottleneck = min(bottleneck, mf.graph[u][idx].residual())
47         v = u
48     v = t
49     while v != s:
50         u, idx = parent_edge[v]
51         e = mf.graph[u][idx]
52         e.flow += bottleneck
53         mf.graph[e.to][e.rev].flow -= bottleneck
54         v = u
55     return bottleneck
56
57
58 def dynamic_maxflow_increase(mf: MaxFlow, s: int, t: int,
59                             edge_loc: Tuple[int, int], k: int) -> int:
60     """ $r > c(u,v)$ : capacity of the edge rises by k. Each unit of extra capacity
61     can raise the max flow by at most 1 (proved in the pset), so at most k new
62     augmenting paths exist. Raise the cap, then try up to k augmentations.
63      $O(k*(V+E))$ ."""
64     u, idx = edge_loc
65     mf.graph[u][idx].cap += k
66     added = 0
67     for _ in range(k):
68         f = _augment_one(mf, s, t)
69         if f == 0:
70             break
71         added += f
72     return mf.value(s)
73
74
75 def dynamic_maxflow_decrease(mf: MaxFlow, s: int, t: int,
76                              edge_loc: Tuple[int, int], k: int) -> int:
77     """ $r < c(u,v)$ : capacity drops by k. If current flow on the edge exceeds the
78     new capacity, we must cancel the excess: reduce the edge's flow and reroute or
79     retract the affected units. We (1) lower cap by k; (2) if flow > cap, cancel
80     the excess along a path back to s and forward from t (flow decomposition),
81     then (3) try to re-augment to recover any flow that can still be routed
82     another way. Each of the  $\leq k$  cancelled units costs  $O(V+E)$ ."""
83     u, idx = edge_loc
84     e = mf.graph[u][idx]
85     e.cap -= k

```

```

86     excess = e.flow - e.cap
87     if excess > 0:
88         # cancel `excess` units through this edge: push them back via residuals.
89         for _ in range(excess):
90             _cancel_one_unit(mf, s, t, u, idx)
91     # after cancellation the flow may be re-augmentable via other routes
92     while _augment_one(mf, s, t) > 0:
93         pass
94     return mf.value(s)
95
96
97 def _cancel_one_unit(mf: MaxFlow, s: int, t: int, u: int, idx: int) -> None:
98     """Remove one unit of flow currently traversing edge (u -> v=graph[u][idx]).
99     Walk from v back to t along flow-carrying edges and from u back to s, undoing
100     one unit, restoring conservation."""
101     e = mf.graph[u][idx]
102     v = e.to
103     # reduce the edge itself
104     e.flow -= 1
105     mf.graph[v][e.rev].flow += 1
106     # retract one unit from v -> ... -> t (follow forward flow edges)
107     cur = v
108     while cur != t:
109         moved = False
110         for ee in mf.graph[cur]:
111             if ee.cap > 0 and ee.flow > 0:
112                 ee.flow -= 1
113                 mf.graph[ee.to][ee.rev].flow += 1
114                 cur = ee.to
115                 moved = True
116                 break
117         if not moved:
118             break
119     # retract one unit from s -> ... -> u (follow forward flow edges into u)
120     cur = u
121     while cur != s:
122         moved = False
123         for w in range(mf.n):
124             for ee in mf.graph[w]:
125                 if ee.to == cur and ee.cap > 0 and ee.flow > 0:
126                     ee.flow -= 1
127                     mf.graph[ee.to][ee.rev].flow += 1
128                     cur = w
129                     moved = True
130                     break
131             if moved:
132                 break
133         if not moved:
134             break
135
136
137 # -----
138 # 7-2: k edge-disjoint paths from sources s_1..s_k to a common target t
139 # -----
140 def edge_disjoint_paths(n: int, directed_edges: List[Tuple[int, int]],
141                        sources: List[int], target: int):
142     """Return k edge-disjoint paths (one per source) to `target`, or None if
143     impossible. Build a flow network with unit-capacity edges, a super-source S
144     connected to each s_i with capacity 1, and max-flow == k iff k disjoint paths
145     exist (integral flow decomposes into unit paths)."""
146     k = len(sources)
147     S = n # super source
148     mf = MaxFlow(n + 1)
149     for (u, v) in directed_edges:
150         mf.add_edge(u, v, 1)
151     for si in sources:
152         mf.add_edge(S, si, 1)
153     flow = mf.max_flow(S, target)
154     if flow < k:
155         return None
156     # decompose: each source sends exactly one unit; extract its path
157     paths = []
158     for si in sources:
159         path = [si]
160         u = si
161         guard = 0
162         while u != target and guard < 2 * (len(directed_edges) + n):
163             guard += 1
164             nxt = None
165             for e in mf.graph[u]:
166                 if e.cap == 1 and e.flow == 1:
167                     nxt = e
168                     break
169             if nxt is None:
170                 break
171             paths.append(nxt)

```

```

172         u = nxt.to
173         path.append(u)
174         paths.append(path)
175     return paths
176
177
178 # -----
179 # 7-3: food-truck assignment (minimize vouchers = maximize matched customers)
180 # -----
181 def food_truck_assignment(customer_prefs: List[List[int]],
182                           quantities: List[int]):
183     """customer_prefs[i] = list of acceptable food-type indices for customer i.
184     quantities[j] = number of servings of food type j.
185     Return (num_assigned, assignment) where assignment[i] = food type given to
186     customer i or None (=> $10 voucher). Minimizing vouchers == maximizing
187     assigned customers == max bipartite b-matching (customers -> foods with
188     capacities), solved by max flow.
189     Nodes: S=0, customers 1..n, foods n+1..n+m, T=n+m+1."""
190     n = len(customer_prefs)
191     m = len(quantities)
192     S = 0
193     T = n + m + 1
194     mf = MaxFlow(n + m + 2)
195     cust_edges = []
196     for i in range(n):
197         loc = mf.add_edge(S, 1 + i, 1)          # each customer once
198         cust_edges.append(loc)
199     food_node = lambda j: 1 + n + j
200     # customer -> acceptable foods (unit)
201     ci_edges = {}
202     for i in range(n):
203         for j in customer_prefs[i]:
204             loc = mf.add_edge(1 + i, food_node(j), 1)
205             ci_edges[(i, j)] = loc
206     for j in range(m):
207         mf.add_edge(food_node(j), T, quantities[j]) # capacity = servings
208     assigned = mf.max_flow(S, T)
209     # read assignment
210     assignment: List[Optional[int]] = [None] * n
211     for (i, j), (u, idx) in ci_edges.items():
212         if mf.graph[u][idx].flow == 1:
213             assignment[i] = j
214     return assigned, assignment
215

```